

데이터 분석 협업 구조

CLIP 01

서브에이전트
구조 이해

CLIP 02

역할 설계
& 구성

CLIP 03

파이프라인
실행하기

CLIP 04

에이전트 실무
운영 팁

데이터 분석 협업 구조

CLIP 01

서브에이전트
구조 이해

CLIP 02

역할 설계
& 구성

CLIP 03

파이프라인
실행하기

CLIP 04

에이전트 실무
운영 팁

이번 챕터에서 만드는 것



만들고 · 관리하고 · 실행하고 · 운영하기



CLIP 01 · 서브에이전트 구조 이해

서브에이전트란

구조 · 특성 · 장단점 · 에이전트 팀 비교

서브에이전트

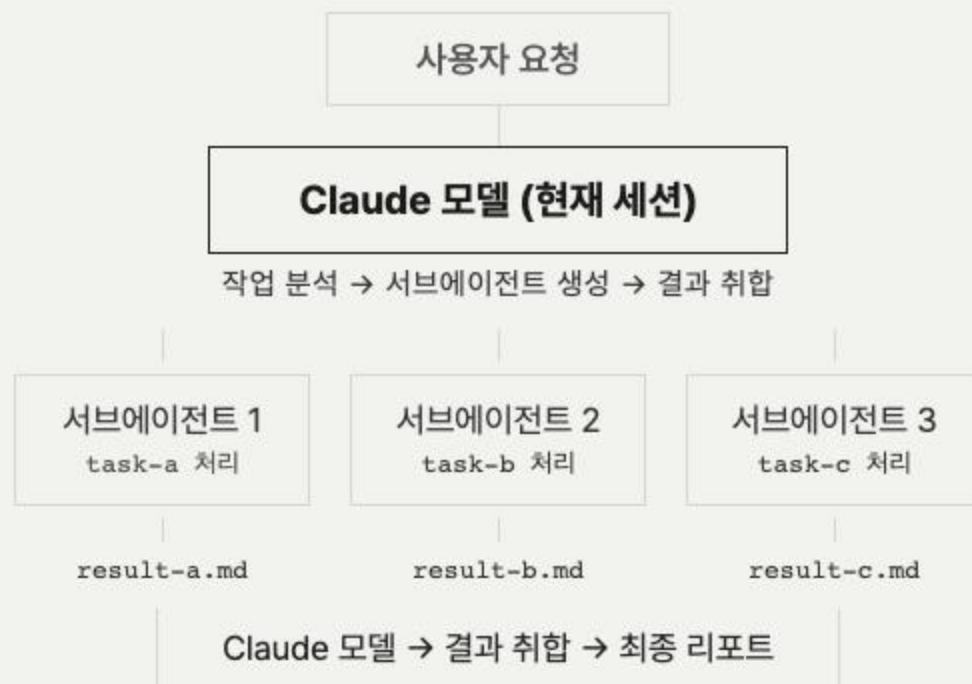
Claude 모델 → 서브에이전트

독립 컨텍스트에서
순차 또는 병렬 실행
파일로 결과 취합

내장 서브에이전트

Explore · Plan · General-purpose

Claude Code에 기본 탑재, 자동 호출
직접 만들면 용도에 맞게 확장 가능



활용 판단

서브에이전트, 언제 쓰면 좋을까

● 이럴 때 쓰면 좋음	● 주의할 점
+ 작업이 쪼개질 때 수집 · 분석 · 시각화처럼 역할이 나뉘는 작업	- 비용이 올라감 에이전트 수만큼 컨텍스트 창 별도 생성
+ 반복 실행이 잦을 때 같은 파이프라인을 매주 돌려야 하는 경우	- 초기 설정 필요 파일 구성·역할 정의에 시간 투자 필요
+ 팀과 공유할 때 .md 파일이라 git으로 팀 전체가 같은 에이전트 사용	- 간단한 작업엔 과함 한 번짜리 질문은 그냥 Claude에게 바로 물어보는 게 빠름

01

독립 실행



자기 일만 받아서 처리
전체 대화는 모름
→ 혼선 없음

- + 집중도 높음 · 역할 명확
- 맥락 따로 전달해야 함

02

순차 또는 병렬 실행



의존성 있으면 순차
없으면 병렬
상황에 맞게 선택

- + 병렬 시 처리 시간 단축
- 에이전트 수만큼 비용 증가

03

파일로만 소통



에이전트끼리 직접 연결 없음
결과 → 파일 저장
메인이 마지막에 수집

- + 결과 추적 · 재실행 쉬움
- 파일 저장 명시적으로 지시해야 함

데이터 시각화에서

서브에이전트의 장단점

데이터 분석 파이프라인
기준 예시

● 장점

- 01 **실패 단계만 재실행 가능**
EDA 완료됐으면 ingestion·EDA 건너뛰고
지표 설계(@analyst)만 다시
- 02 **역할별 독립 컨텍스트가 있어 혼선 방지**
@metrics는 탐색만, @analyst는 지표만
역할 혼선 없음
- 03 **병렬 처리도 선택적으로 가능**
CSV 여러 파일 동시 분석처럼
독립 작업은 한꺼번에

● 단점

- 01 **초기 설정 필요**
에이전트 .md 파일 있어야 함
- 02 **단순 작업엔 오버스펙**
짧은 작업은 한번에 요청이 빠름
- 03 **에이전트 간 직접 소통 불가**
파일 경유만 — 유연성 제한

에이전트 등록 위치

`CLAUDE.md`

메인 지시서

누가 언제 어떤 에이전트를
호출할지 지시

`.claude/agents/`

프로젝트 전용

git으로 팀 전체 공유 가능

`~/.claude/agents/`

개인 에이전트

내 모든 프로젝트에서 사용 가능

이렇게 쓰면 됨

`@metrics` 탐색 분석해줘

프로젝트 전용

my-project/

├─ `CLAUDE.md`└─ `.claude/` # git으로 팀 공유└─ `agents/`└─ `eda.md`└─ `metrics.md`

개인 에이전트 (모든 프로젝트)

~/.claude/

└─ `agents/` # 어디서든 @로 호출└─ `code-reviewer.md`

에이전트 설정

에이전트 파일 작성법

1 이름 + 설명

`name` — @로 호출할 이름`description` — 한 줄 역할 요약

2 모델 + 도구 (둘 다 선택)

`model` — 없으면 세션 모델 상속`tools` — 없으면 모든 도구 허용

3 역할 지시서

--- 아래에 자연어로 작성

무엇을 · 어떻게 · 어디에 저장할지

name + description 두 필드만 필수

`.CLAUDE/AGENTS/METRICS.MD`

`name:eda` ← ● @metrics 로 호출`description:탐색적 분석 및 시각화 전담` ← ● 역할 요약`model:haiku` ← ● 선택`tools:Bash, Read, Write` ← ● 선택

데이터를 탐색 분석하고 ← ● 역할 지시서

outputs/metrics.md 에 저장한다.

에이전트 생성

두 가지 만드는 방법

에디터와 터미널,
방식이 달라요

name + description

두 필드만 필수

방법 1 — 코드에디터

안티그래비티 · VS Code

Claude에게 생성 요청
/agents 명령 사용 불가

1 Claude에게 프롬프트

"데이터 분석 에이전트 생성해줘"

2 .claude/agents/ 파일 자동 생성

name · description 포함

3 Claude Code 세션 재시작

→ @analyst 즉시 호출 가능

세션 재시작 필요

방법 2 — 터미널

터미널

/agents 슬래시 명령
인터랙티브 UI로 생성

1 터미널에서 입력

/agents

2 이름 · 역할 입력

UI 안내에 따라 작성

3 저장 즉시 사용 가능

재시작 불필요

.md 파일 자동 저장 → git 관리 가능

실습해 보기

서브에이전트 3명 구성

역할만 설명하면

.md 파일 자동 생성

복붙해서 바로 써도 됨

요청

"데이터 분석용 서브에이전트
3개 만들어줘 - EDA, 지표설계,
시각화 역할로"

.CLAUDE/AGENTS/ 에 생성됨

```
|— metrics.md 탐색적 분석 · 이상값 탐지
|— metrics.md 지표 설계 · 기준값 정의
|— visualizer.md 차트 생성 · 시각화
```

호출

```
@metrics / @metrics / @visualizer
```

이번 챕터에서 만드는 것



만들고 · 관리하고 · 실행하고 · 운영하기



CLIP 02 · 에이전트 팀 구성

역할 설계 & 구성

CLAUDE.md 작업 흐름 · 폴더 구조 · 에이전트 역할 설계

CLAUDE.MD 흐름 설계

에이전트 관리 CLAUDE.md

역할 분담 → 에이전트

실행 순서 → CLAUDE.md

@에이전트명 + 순서만 써도

메인이 그대로 실행

에이전트별 모델도

여기서 지정해요

CLAUDE.md

AI 데이터 분석 파이프라인

7개의 전문 AI 에이전트를 순서대로 지휘합니다.

파이프라인

@eda → @metrics → @visualizer

단계	에이전트	모델	산출물
1	@eda	haiku	01_profile.md
2	@metrics	sonnet	02_metrics.md
3	@visualizer	sonnet	03_report.md

실행 규칙

- 반드시 순서대로
- 이전 단계 파일이 있어야 다음 단계 시작
- 실패 시 해당 단계에서 중단 후 보고

실습해 보기

CLAUDE.md에 흐름 쓰기

에이전트 순서와

모델을 CLAUDE.md에

자연어로 작성

순서 + 모델 한 번만 써두면

매번 다시 지시 불필요

에이전트 한번에 묶으려면?

다음 클립에서 /run-task

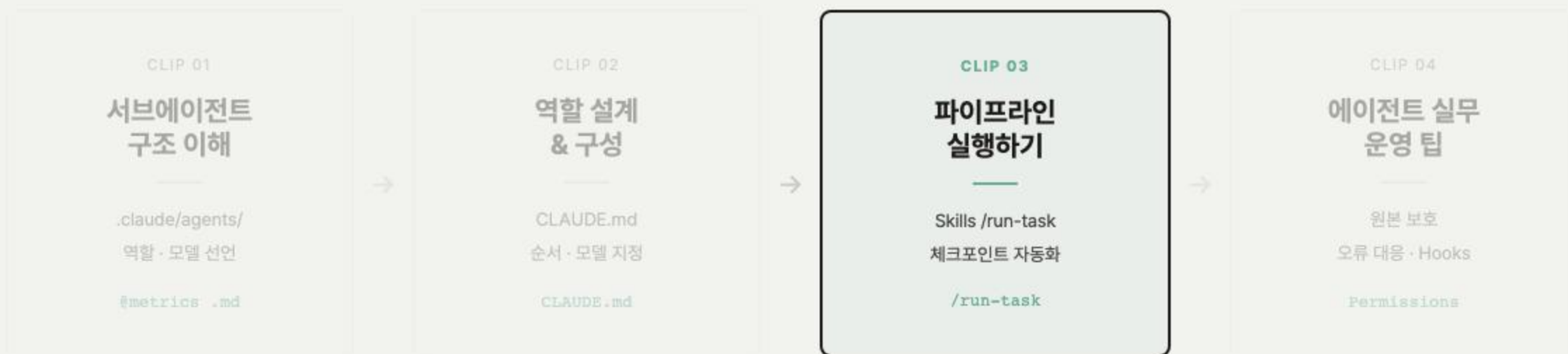
CLAUDE에게 요청

"이 에이전트들 관리하는 CLAUDE.md 만들어줘

순서: @eda → @metrics → @visualizer

모델: eda는 haiku, 나머지는 sonnet"

이번 챕터에서 만드는 것



만들고 · 관리하고 · 실행하고 · 운영하기



CLIP 03 · 실행 & 병렬 처리

파이프라인 실행하기

Skills 파일 만들기 · 체크포인트 · 실습

스킬을 만드는 이유

프롬프트 vs /run-task

매번 설명하는 것과
한 번 만들어두는 것의
차이

	그냥 프롬프트	/run-task 스킬
실행	"eda-metrics-visualizer 순서로 실행해줘"	/run-task 실행해줘
체크포인트	✕ 없음	✓ 완료 단계 자동 skip
실패 대응	처음부터 다시	실패 단계부터 재시작
재현성	매번 다시 설명	항상 동일한 규칙
팀 공유	대화에만 존재	파일로 git 공유

실습해 보기

/run-task 스킬 만들기

파일 하나면

/run-task 명령어가

생겨요

완료된 단계는

outputs/ 파일로 확인

파일 있으면 skip

없으면 실행

CLAUDE에게 요청

"run-task 스킬 파일 만들어줘

순서: @eda → @metrics → @visualizer

각 단계 완료 후 outputs/에 마커 파일 저장

재실행 시 마커 있으면 skip, 없으면 실행

실패 시 어느 단계인지 알려주고 중단"

생성되는 파일 구조

스킬 파일

```
.claude/
├── skills/
│   └── run-task/
│       └── SKILL.md
```

체크포인트 출력물

```
outputs/ # 파일 있으면 skip
├── eda.md ✓
├── metrics.md ✓
└── visualizer.md ⚠
```

실습해 보기

분석 실행하기

`/run-task`

입력 파일만 넣으면

서브에이전트들이 순서대로 실행

실패 단계만 서브에이전트 재호출 가능

실행

`"/run-task 실행해줘"`

실행 흐름

STEP 1 @eda

데이터 수집



↓

STEP 2 @metrics

탐색 분석



↓

STEP 3 @visualizer

리포트 생성



 파이프라인 완료

이번 챕터에서 만드는 것



만들고 · 관리하고 · 실행하고 · 운영하기



CLIP 04 · 실무 운영

에이전트 실무 운영 팁

원본 보호 · 오류 대응 · 자동화

원본 데이터 보호

써도 되는 곳만 열어두기

원본 CSV는 잠그고
분석 결과 폴더만
쓰기 허용

① 이렇게 요청 하면

원본 data/raw는 쓰기 금지,
processed·charts·reports만 허용.
git push는 매번 확인하게 해줘

자동 생성 ↓

② .CLAUDE/SETTINGS.JSON

```
"permissions": {
  "allow": ["Write(processed/**)",
            "Write(charts/**)",
            "Write(reports/**)"],
  "ask":   ["Bash(git push *)"],
  "deny":  ["Write(data/raw/**)"]
}
```

deny — 잠금

원본 폴더 보호

덮어쓰기 차단

실수로 날리면 복구 불가

ask — 매번 확인

위험하지만 필요한 명령

git push · 차트 외부 업로드

막진 말되 매번 사람 승인

allow — 자유 허용

작업용 폴더만 쓰기 허용

원본은 그대로 두고

결과 만드는 폴더에서만 자유롭게

파이프라인 운영

잘 돌리는 3가지 팁

파이프라인이 길어질수록

차이가 나는

일상 운영 습관

TIP 1 · 대화 관리

/compact 으로 방향 지정 압축

`"/compact` 분석 결과 유지,
`시행착오 버려"`

TIP 2 · 실행 제어

@에이전트 이름으로 콧 집어 호출

`"@visualizer` 차트 다시 만들어줘"

TIP 3 · 비용·속도

에이전트별 모델 따로 지정

단순 EDA는 `Haiku`, 깊은 분석은 `Sonnet`

차트 자동 확인

사람 대신 챙겨주기

차트가 완성되면

자동으로 화면에 띄워

바로 눈으로 확인

① 이렇게 요청하면

charts 폴더에 차트 파일 생길 때마다
자동으로 열어주는 혹 추가해줘

자동 생성 ↓

② .CLAUDE/SETTINGS.JSON

```
"hooks": { "PostToolUse": [{  
  "matcher": "Write",  
  "command": "open charts/*.png"  
}]}
```

실행 시 ↓

③ 이렇게 작동

차트 그릴 때마다 자동으로 열림

매번 폴더 열어서 확인할 필요 없음